EE2-15

IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2012

ISE PART II: MEng, BEng and ACGI

**LANGUAGE PROCESSORS**

Corrected Copy
Q1 f numbering!

Wednesday, 30 May 2:00 pm

Time allowed: 1:30 hours

**There are THREE questions on this paper.**

**Answer ALL questions.**
**Q1 carries 40% of the marks. Questions 2 and 3 carry equal marks (30% each).**

**Any special instructions for invigilators and information for candidates are on page 1.**

Examiners responsible First Marker(s) : Y.K. Demiris
Second Marker(s) : J.V. Pitt

© Imperial College London

## The Questions

### Answer ALL questions

1.

(a) Provide the formal definition of a Turing Machine. Explain what conditions need to be satisfied for a Turing Machine to be converted into a Linearly-bounded Automaton (LBA). [10]

(b) In the context of lexical analysis, once the conversion from Non-deterministic Finite Automaton (NFA) to Deterministic Finite Automaton (DFA) has been completed, the resulting DFA might not be minimal. Provide the DFA minimization algorithm. [6]

(c) Provide a class of grammars that are known to be ambiguous and give an example of a common ambiguous grammar fragment. [6]

(d) Provide the definition of S-attributed grammars, and explain how the parser stack can be modified to allow evaluation of their attributes during bottom up parsing. [6]

(e) Explain why the grammar below is not LL(1), and use the left-factoring procedure to transform it to its LL(1) equivalent.

Y -> xYa
Y -> xYb
Y -> cd

[4]

(f) Calculate the FIRST and FOLLOW sets for all non-terminal symbols for the grammar below where {if, a, b, then, else} are terminal tokens, and {S, S', E} are non-terminals:

(1) S -> if E then S S'
(2) S -> a
(3) S' -> else S
(4) S' -> ε
(5) E -> b

[8]

2. You are required to construct the minimal deterministic finite state automaton (DFA) for the regular expression $(a|b)^*c^*$ following the steps below. You should *clearly mark all final states* in *all* the automata you construct in the process.

(a) Construct a non-deterministic finite automaton (NFA) using Thompson's algorithm. [10]

(b) Construct the equivalent DFA using the subset construction algorithm. *Explain the intermediate steps you have taken.* [10]

(c) Apply the DFA minimization algorithm to the DFA you have constructed in (b). Show whether your DFA was already minimal or not. *Explain clearly the intermediate steps of the application of the DFA minimization algorithm.* [10]

3.

(a) In the context of top-down parsing, the calculation of FIRST and FOLLOW sets are used in the construction of parsing tables. Provide the algorithm for constructing a top-down parsing table. [14]

(b) Construct the parsing table for the grammar in Question 1(f) using the table below (and the FIRST and FOLLOW sets you calculated in Q 1(f)), with $ denoting the end of input marker. Complete *ALL* table entries, clearly marking any error entries (two examples shown).

[16]

| Non-terminal | Input Token | | | | | |
|---|---|---|---|---|---|---|
| | a | b | else | if | then | $ |
| S | | | | | | |
| S' | Error | | | | | |
| E | | | Error | | | |

*(NB: Make sure you copy this table to your exam booklet!)*

E2.15: Language Processors
Sample answers to exam questions 2012

Question 1

(a) [bookwork] A Turing Machine M is defined as $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$ where

- Q: a finite set of N states $q_0, q_1, \ldots, q_N$ of the finite control
- $\Sigma$: a finite input alphabet of symbols
- $\Gamma$: the complete set of tape symbols, $\Sigma \subseteq \Gamma$
- $\delta(q,X)$: the transition function between states. Given a state $q \in Q$, and a tape symbol $X \in \Gamma$, the function $\delta(q,X)$ returns a triplet (p,Y, D), where p is the new state in Q, $Y \in \Gamma$ is the symbol that replaces the current symbol on the tape, and D is either {left,right} indicating where will the head move next
- $q_0$: the start state
- B is the blank symbol ($B \in \Gamma$ but $B \notin \Sigma$) initially written in all tape cells except the ones holding the input
- F: the set of final states, $F \subseteq Q$

[NB: the students need not use the symbols used above – any will do; however they are expected to be exact as to what constitutes a Turing Machine, and incorporate all of the elements above].

LBA are a restricted class of TMs wrt the length of the tape, which in LBAs is not infinite, but proportional to the length of the input string. LBAs are deterministic, and are guaranteed to give a decision as to whether the input string is legal or not in an amount of time proportional to the length of the input string.

(b) [Bookwork] In order to minimise the DFA we obtained from the subset construction algorithm, we start by assuming that all states of the DFA are equal, and we work through the states, putting different states in separate sets if (a) one is final and other is not (b) the transition function maps them to different states, based on the same input character. The DFA minimization algorithm proceeds by initially creating two sets of states, final and non-final – non-final: {A, B, C} and final: {D}. For each state set created, the algorithm examines the transitions for each state and for each input symbol.

(c) [Bookwork + new computed example]

A class of grammars known to be ambiguous are those that contain rules that have both left and right recursion. Most common ambiguous grammar fragment is the one related with the if_then_else statements, for example:

```
Statement -> if Expression then Statement else Statement
           | if Expression then Statement
           | other
```

(d) [Bookwork]

S-attributed grammars are those that do not contain any inherited attributes, i.e. all attributes of a node can be calculated (synthesised) by the attributes of its children nodes. We can use extra fields in the parser stack to hold the values of the synthesized attributes; each child note stacks it synthesized attributes and when a reduce takes place, the parent node them pops them up and processes them and puts the result back to the stack.

(e) [Bookwork]

The grammar is not LL(1) since for there is a common prefix in more than one production rules for the same non-terminal (Y), i.e. rules 1 and 2 have common xY as a prefix. One symbol lookahead (LL(1)) will not assist in the choice of which rule to apply.

To convert it to LL(1) we use left factoring

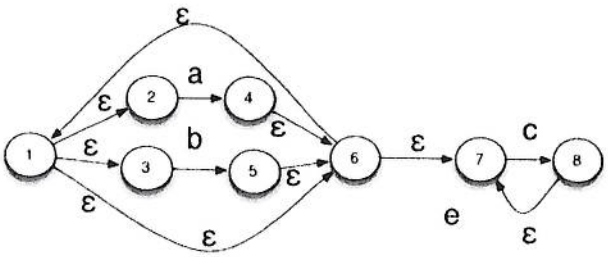
Y -> xYK
K -> a | b
Y -> cd

(f)

(g) [New Computed Example]:
FIRST(S) = { if, a}
FIRST(S')= {else, ε}
FIRST(E) = {b}
FOLLOW(S) = {$, else};
FOLLOW(S') = {$, else}
FOLLOW(F) = {then}

***Question 2*** [new computed example]

(a) Applying Thompson' algorithm you get




End states: 1, 6, 8 (7, and ALL are also valid answers)

(b) Applying the subset construction algorithm on this NFA we get:

ε-closure(StartState) = ε-closure{1} = {1, 2, 3, 6, 7} = A

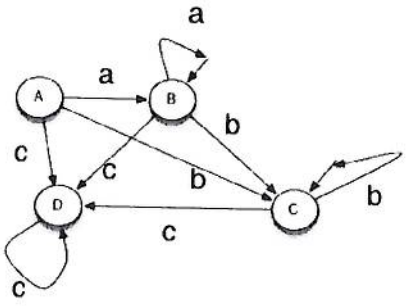
ε-closure(move(A, a)) = ε-closure{4} = {4, 6, 1, 2, 3, 7} = B
ε-closure(move(A, b)) = ε-closure{5} = {5, 6, 7, 1, 2, 3} = C
ε-closure(move(A, c)) = ε-closure{8} = {7, 8} = D

ε-closure(move(B, a)) = ε-closure{4} = {4, 6, 1, 2, 3, 7} = B
ε-closure(move(B, b)) = ε-closure{5} = {5, 6, 7, 1, 2, 3} = C
ε-closure(move(B, c)) = ε-closure{8} = {7, 8} = D

ε-closure(move(C, a)) = ε-closure{4} = {4, 6, 1, 2, 3, 7} = B
ε-closure(move(C, b)) = ε-closure{5} = {5, 6, 7, 1, 2, 3} = C
ε-closure(move(C, c)) = ε-closure{8} = {7, 8} = D

ε-closure(move(D, a)) = ε-closure(move(D, b)) = {}
ε-closure(move(D, c)) = ε-closure{7} = {7,8} = D

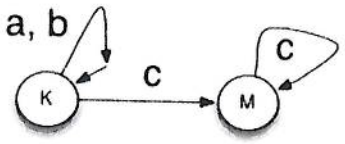
No more new states, we are done. The resulting DFA is below. ALL states are final states.




(c) In order to minimise the DFA we obtained from the subset construction algorithm, we start by assuming that all states of the DFA are equal, and we work through the states, putting different states in separate sets if (a) one is final and other is not (b) the transition function maps them to different states, based on the same input character.

The DFA minimization algorithm proceeds by initially creating two sets of states, final and non-final – in our case they are all final. For each state set created, the algorithm examines the transitions for each state and for each input symbol.

In our case this set is further subdivided in two sets, one with {A, B, C}, and one with {D} since the three first ones go to the same states for the same symbols. If we name the first set {A, B, C} as K, and M = {D}, and draw all transitions, we get the following minimal DFA:




---

Question 3: [New computed example]

(a) *For each production rule of the form A -> α do:*

- *For each terminal x in FIRST(α), add A -> α to M[A, x]*
- *If FIRST(α) contains ε, add A -> α to M[A, b] for each terminal b in FOLLOW(A)*
- *If FIRST(α) contains ε, and FOLLOW(A) contains $, add A -> α to M[A, $]*
- *Mark all undefined entries of M as "error".*

(b)

| Non-terminal | Input Token | | | | | |
|---|---|---|---|---|---|---|
| | a | b | else | if | then | $ |
| S | S->a | Error | Error | S->iEtSS' | Error | Error |
| S' | Error | Error | S' -> ε<br>S'->eS | Error | Error | S'-> ε |
| E | Error | E->b | Error | Error | Error | Error |

1 mark for each correct entry on this table (total 16 (2 given))